

Instructions. Answer each of the following questions. For multiple-choice questions, mark the *one* best answer. For fill-in-the-blank questions, give a single exact value or expression. When a question asks for privacy cost, give the value of ϵ . For sensitivity and privacy cost answers, feel free to provide the *expression* for calculating the answer—there is no need to actually perform the arithmetic. Unless a question says otherwise, assume that each individual’s data is contained in exactly one row of the data, and that two datasets are neighbors if one can be obtained from the other by adding or removing one row. Unless a question says otherwise, the quasi-identifiers are exactly the columns named in the question. In the **adult** dataset and in the tables below, the **Target** column records a person’s income ($\leq 50K$ or $>50K$). For questions that contain code, refer to the definitions on the last page of the exam.

1. (5 points) An attacker has two dataframes: **aux**, with columns **Name**, **Zip**, and **Sex** for the residents of a town, and **released**, a de-identified dataset with columns **Zip**, **Sex**, and **Diagnosis**. Which expression performs a linking attack that pairs names with diagnoses?

- ☐ `pd.merge(aux, released, left_on=['Name', 'Zip'], right_on=['Name', 'Zip'])`
☐ `pd.concat([aux, released], axis=1)`
☐ `pd.merge(aux, released, left_on=['Zip', 'Sex'], right_on=['Zip', 'Sex'])`
☐ `released.groupby(['Zip', 'Sex'])['Diagnosis'].count()`

2. (5 points) An attacker has the auxiliary data **aux** and the de-identified data **deid** shown below. The six rows of **deid** belong to six different people. The attacker runs:

```
pd.merge(aux, deid, left_on=['Zip'], right_on=['Zip'])
```

How many rows does the result contain?

aux			deid		
Name	DOB	Zip	DOB	Zip	Target
Alice	1990-03-14	05401	1990-03-14	05401	$\leq 50K$
Bob	1985-07-02	05401	1990-03-14	05401	$>50K$
Carol	1990-03-14	05403	1985-07-02	05401	$>50K$
Dan	1972-11-30	05405	1990-03-14	05403	$\leq 50K$
			1972-11-30	05405	$\leq 50K$
			1999-01-05	05405	$>50K$

Number of rows: _____

3. (5 points) Using the tables in Question 2, the attacker instead joins on *both* columns:

```
pd.merge(aux, deid, left_on=['DOB', 'Zip'], right_on=['DOB', 'Zip'])
```

Which people in **aux** are *uniquely* re-identified by this attack (that is, matched to exactly one row of **deid**)?

Names: _____

4. (5 points) An attacker first links on ['DOB'] and then repeats the attack on ['DOB', 'Zip']. Assume the auxiliary data is accurate and that every person in it appears in the de-identified data. Compared with the first attack, the second attack:
- ☐ returns more rows, because each person now has two columns on which a de-identified row can match, so more rows count as matches.
 - ☐ returns the same number of rows or fewer and uniquely re-identifies at least as many people, but requires auxiliary data that includes ZIP codes.
 - ☐ uniquely re-identifies fewer people, because a de-identified row must now agree on two columns instead of one, so some true matches are lost.
 - ☐ returns exactly the same rows, because a person's date of birth already determines their ZIP code in any dataset.
5. (5 points) In a de-identified hospital dataset with 10,000 rows (one per patient), 9,000 patients are uniquely identified by their ZIP code. An attacker knows a target's name and ZIP code, and knows that the target is a patient in the dataset. Which statement is correct?
- ☐ The attacker can single out the target's row only if the attacker also knows the target's date of birth.
 - ☐ If the target is one of the 9,000, a join on ZIP code alone recovers the target's entire row, including the diagnosis.
 - ☐ The attacker learns nothing, because a ZIP code by itself is not identifying information once names are removed.
 - ☐ The attacker can single out all 10,000 rows, because almost every row in the dataset has a unique ZIP code.
6. (5 points) An analyst may see only aggregate statistics about a table that includes a row for Ardyce Golby. The analyst learns that:
- the table has 40 rows, and the mean **Education-Num** over all 40 rows is 10.1;
 - the 39 rows whose **Name** is not 'Ardyce Golby' have mean **Education-Num** 10.0.

Using a differencing attack, what is Ardyce's **Education-Num**?

Ardyce's **Education-Num**: _____

7. (5 points) An attacker knows that Karrie Trusslove has exactly one row in the **adult** dataset. The dataset's **Target** column has only two values, '<=50K' and '>50K'. The attacker runs two counting queries:

```
q1 = len(adult[adult['Target'] == '>50K'])
q2 = len(adult[(adult['Target'] == '>50K') &
               (adult['Name'] != 'Karrie Trusslove')])
```

Both queries return 1204. What is Karrie's **Target** value?

Karrie's **Target**: _____

8. (5 points) Each release below contains only statistics computed over groups of more than 1,000 residents of a city. Which release allows a differencing attack that reveals one resident's income?
- ☐ The average income of all residents, and the median income of all residents.
 - ☐ The total income of all residents, released once.
 - ☐ The total income of the residents of each county, where every county has over 1,000 residents.
 - ☐ The total income of all residents, and the total income of all residents except the mayor.

9. (5 points) A table satisfies 4-Anonymity but does *not* satisfy 5-Anonymity for a set of quasi-identifiers. How many rows are in its smallest group?

Size of the smallest group: _____

10. (5 points) The output of `df[['Age', 'Zip']].value_counts()` for a generalized table `df` with quasi-identifiers `Age` and `Zip` is:

Age	Zip	
30	5400	12
40	5400	9
30	5600	7
40	5600	3
50	5600	3

Without changing any values, what is the smallest number of rows that must be removed so that the remaining table satisfies 5-Anonymity?

Number of rows: _____

11. (5 points) A table's only quasi-identifier is `Age`. After generalizing to depth 1, every generalized value (10, 20, ..., 80) appears in at least 20 rows, so the table satisfies 20-Anonymity; no one is 90 or older. Then one new row with `Age` 105 is added. Which change restores 20-Anonymity while keeping the most information about age?
- ☐ Generalize `Age` to depth 2 instead of depth 1.
 - ☐ Collect more rows of data from people aged 20 to 60.
 - ☐ Clip ages to at most 89 before generalizing to depth 1.
 - ☐ Generalize `Age` to depth 3 instead of depth 1.
12. (5 points) In a dataset, most people are between 20 and 50 years old, but a few are between 80 and 90. `Age` is the only quasi-identifier, and `generalize` applies the same depth to every row. Why is it hard to choose a good depth?
- ☐ `generalize` cannot handle ages of 80 or more, so those rows must be removed.
 - ☐ The large groups of people aged 20–50 violate k -Anonymity unless they are split.
 - ☐ Each depth creates groups of equal size, so the few older people cannot fill a group.
 - ☐ Any depth that gives the older ages k rows per group over-generalizes the younger ages.
13. (5 points) Which of the following queries does *not* have a global sensitivity of 1?
- ☐ `len(adult[adult['Occupation'] == 'Sales'])`
 - ☐ `adult[adult['Age'] > 60]['Age'].sum()`
 - ☐ `len(adult[adult['Age'] > 60])`
 - ☐ `len(adult[(adult['Age'] > 60) & (adult['Sex'] == 'Female')])`

14. (5 points) On a particular dataset, `query2()` (the number of participants over the age of 60) returns 4,000. A student claims that the sensitivity of `query2` is 4,000, because removing all of the participants over 60 would change the answer by 4,000. Which is the best explanation of the student's error?
- ☐ The sensitivity should be the number of rows in the whole dataset, since any row could be removed.
 - ☐ The student is correct: sensitivity is the change in the answer when everyone who is counted is removed.
 - ☐ The sensitivity should be 0, because removing a participant who is not over 60 does not change the answer.
 - ☐ Neighboring datasets differ in one row, so sensitivity measures the effect of adding or removing one row, which is at most 1 for a count.
15. (5 points) The same counting query is run on two datasets: dataset A has 100 rows and dataset B has 1,000,000 rows. Which statement about the query's global sensitivity is correct?
- ☐ It is 1/100 for dataset A and 1/1,000,000 for dataset B.
 - ☐ It is larger for dataset B, because B has more rows that could be added or removed.
 - ☐ It is 1 for both datasets, because adding or removing one row changes any count by at most 1.
 - ☐ It is larger for dataset A, because one row is a larger fraction of a small dataset.
16. (5 points) An analyst calls `dp_query1(0.1)` twice on the same dataset and gets two different answers. Which explanation is correct?
- ☐ The Laplace mechanism is randomized, so each call adds a newly sampled amount of noise to the same exact count.
 - ☐ The function has a bug, because a differentially private query should return the same answer every time.
 - ☐ The dataset changed between the calls, because differential privacy removes a random row each time.
 - ☐ The sensitivity is recomputed from the data on each call, so the scale of the noise changes.
17. (5 points) A student sees that the errors for $\epsilon = 1.0$ were tiny, and suggests using $\epsilon = 10$ instead "to make the results even better." Which is the best response?
- ☐ It would increase the error, because a larger ϵ adds more noise.
 - ☐ It would reduce the error further, but outputs could then differ much more with or without one person's data, so privacy would be much weaker.
 - ☐ It would give the same privacy as $\epsilon = 1.0$, because the sensitivity of the count is still 1.
 - ☐ It would give the same error as $\epsilon = 1.0$, because the error has already reached its minimum.
18. (5 points) A hospital uses the Laplace mechanism with $\epsilon = 0.1$ to release two counts: the number of patients treated for flu (true answer 8,000) and the number treated for a rare disease (true answer 6). Which outcome is most likely?
- ☐ The flu count will have a small percent error, while the rare-disease count may be too noisy to be useful.
 - ☐ Both counts will have about the same percent error, because both use the same ϵ .
 - ☐ The rare-disease count will be more accurate, because a small count needs less noise.
 - ☐ The flu count will be less accurate, because a count of 8,000 has a larger sensitivity than a count of 6.

19. (5 points) The query `raw_sum()` below sums a column *without* clipping. Which statement about its global sensitivity is correct?

```
def raw_sum():  
    return adult['Capital Gain'].sum()
```

- ☐ It is 1, because adding or removing one row adds or removes only one term of the sum.
 - ☐ It is the largest capital gain in `adult`, because no row in the data has a larger value.
 - ☐ It is unbounded, because a neighboring dataset could add a row with an arbitrarily large capital gain.
 - ☐ It is the number of rows in `adult`, because every row contributes one term to the sum.
20. (5 points) The following function is supposed to satisfy ϵ -differential privacy, but it does not. Which change fixes it?

```
def dp_sum_age(epsilon):  
    b = 125  
    clipped_sum = adult['Age'].clip(lower=0, upper=b).sum()  
    return laplace_mech(clipped_sum, 1, epsilon)
```

- ☐ Change the sensitivity argument of `laplace_mech` from 1 to `b`.
 - ☐ Change `upper=b` to `upper=adult['Age'].max()` in the call to `clip`.
 - ☐ Change the last argument of `laplace_mech` from `epsilon` to `epsilon/2`.
 - ☐ Remove the call to `clip`, so that none of the ages are changed.
21. (5 points) An analyst is deciding between clipping parameters $b = 10,000$ and $b = 20,000$ for a differentially private sum, with the same ϵ either way. Suppose every value in the column is below 10,000. Compared with $b = 10,000$, what should the analyst expect from $b = 20,000$?
- ☐ Less error, because a larger clipping parameter includes more of the data in the sum.
 - ☐ The same error, because clipping changes none of the values with either parameter.
 - ☐ A larger privacy cost, because doubling the sensitivity doubles the value of ϵ .
 - ☐ More error, because the scale of the noise doubles while no additional data is included.
22. (5 points) Two analysts compute a differentially private sum of the same column, both with $\epsilon = 1$. The column's values are spread between 0 and 50,000. Analyst A uses clipping parameter $b = 100$; Analyst B uses $b = 100,000$. Which comparison is most accurate?
- ☐ A adds less noise but adds significant bias due to clipping; B adds no bias due to clipping but adds 1,000 times as much noise.
 - ☐ A adds more noise than B, because clipping more of the values makes the sum more sensitive to each row.
 - ☐ A and B have the same amount of error, because the error is determined by ϵ , which is the same for both.
 - ☐ B has a larger privacy cost than A, because B's query has a larger sensitivity than A's query.

23. (5 points) Which argument correctly shows that `dp.avg.capgain(epsilon)` (last page) has a total privacy cost of `epsilon`?
- ☐ By parallel composition the total is ϵ , because the sum uses the **Capital Gain** column while the count only counts rows.
 - ☐ By sequential composition the noisy sum and the noisy count cost $\epsilon/2 + \epsilon/2 = \epsilon$, and the division is post-processing.
 - ☐ By post-processing the total is $\epsilon/2$, because the quotient is computed from two values that each have a privacy cost of $\epsilon/2$.
 - ☐ By sequential composition the total is $3\epsilon/2$, because the noisy sum, the noisy count, and their quotient are three releases.

Definitions

The dataframe `adult` is the census dataset from the homework. It has one row per person and columns including `Name`, `DOB`, `SSN`, `Zip`, `Age`, `Sex`, `Occupation`, `Education-Num`, `Capital Gain`, and `Target` ('<=50K' or '>50K').

```
# Generalizing a column to a depth of n replaces the n least-significant  
# digits of each number in that column by zeroes.  
def generalize(df, depths):  
    return df.apply(lambda x: x.apply(lambda y:  
        int(int(y/(10**depths[x.name]))*(10**depths[x.name]))))  
  
def laplace_mech(v, sensitivity, epsilon):  
    return v + np.random.laplace(loc=0, scale=sensitivity / epsilon)  
  
def pct_error(orig, priv):  
    return np.abs(orig - priv)/orig * 100.0  
  
def dp_sum_capgain(epsilon):  
    b = 200000  
    clip_sum = adult['Capital Gain'].clip(lower=0, upper=b).sum()  
    noisy_sum = laplace_mech(clip_sum, b, epsilon)  
    return noisy_sum  
  
def dp_avg_capgain(epsilon):  
    noisy_sum = dp_sum_capgain(epsilon/2)  
    count = len(adult)  
    noisy_count = laplace_mech(count, 1, epsilon/2)  
    return noisy_sum / noisy_count
```

The questions also refer to the following functions from the homework:

- `query1()` returns the number of participants who have never been married.
- `query2()` returns the number of participants who are over the age of 60.
- `dp_query1(epsilon)` and `dp_query2(epsilon)` return differentially private answers to `query1()` and `query2()`, computed with `laplace_mech` and the privacy parameter `epsilon` (except where a question shows a different definition).

Pandas functions used in this exam:

- `pd.merge(left, right, left_on=cols, right_on=cols)` joins two dataframes. The result has one row for every pair of a row of `left` and a row of `right` that agree on all of the columns in `cols`; a row with no match does not appear in the result.
- `pd.concat([df1, df2])` stacks the rows of `df1` and `df2`; with `axis=1` it places them side by side, lining up rows by their position (index).
- `s.value_counts()` returns, for each distinct value in the series `s`, the number of rows with that value. For a dataframe, `df[cols].value_counts()` counts each distinct combination of values in the columns `cols`.
- `df.groupby(cols)['C'].count()` returns the number of rows in each group of rows that share the same values for `cols`.
- `len(df)` returns the number of rows in the dataframe `df`, and `df[cond]` keeps only the rows for which `cond` is true.
- `s.clip(lower=a, upper=b)` replaces each value of the series `s` that is less than `a` by `a`, and each value that is greater than `b` by `b`.